

Department of Computer and Software Engineering
SE: Machine Learning

Course Instructor: Dr. Ahmed Raza	Date: 26-February-2026
Day: Thursday	Semester: 6th

Lab 3: Hard Margin SVM — Geometry of Maximum Margin

Lab Title	CLO	BT	PLO	Weightage
Hard Margin SVM: Geometry of Maximum Margin				

Name	Roll Number	Report /15	Viva /5	Total /20
Zunaira Abdul Aziz	BSSE23058			

Checked on: _____

Signature: _____

1 Learning Outcomes

After completing this lab, the student will be able to:

- Explain the maximum margin principle.
- Compute geometric margin width.
- Verify hard-margin constraints numerically.
- Identify support vectors and interpret their role.
- Analyze geometric sensitivity to scaling and point removal.

2 Equipment Required

- PC with Python 3.x
- NumPy
- Matplotlib
- scikit-learn

3 Theory and Background

Hard Margin SVM solves:

$$\min_{w,b} \frac{1}{2} \|w\|^2$$

Subject to:

$$y_i(w^T x_i + b) \geq 1$$

Geometric margin width:

$$\frac{2}{\|w\|}$$

Support vectors satisfy:

$$y_i(w^T x_i + b) = 1$$

Only these points determine the separating boundary.

4 Part A — Training a Hard Margin Classifier

Task A1: Generate Separable Dataset (2 Marks)

Implementation Box

Generate a 2D linearly separable dataset using a fixed random seed.
Plot the dataset with labels.

```
def generate_dataset(n=100, seed=42):  
    np.random.seed(seed)  
    X = np.random.randn(n, 2)  
    X[50:] += 5  
    y = np.array([-1] * 50 + [1] * 50)  
    return X, y
```

Task A2: Train Linear SVM (2 Marks)

Implementation Box

Train a linear SVM using sklearn:
SVC(kernel='linear')
This model will compute the maximum-margin separating hyperplane.

```
def train_linear_svm(X, y):  
    model = SVC(kernel="linear", C=1e10)  
    model.fit(X, y)  
    return model
```

Task A3: Extract Parameters (2 Marks)

```

Analysis Box

Extract w and b.
Print ||w||

def extract_parameters(model):
    w = model.coef_[0]
    b = model.intercept_[0]
    return w, b

... ||w||: 0.5497

```

5 Part B — Verifying Hard Constraints

Task B1: Constraint Verification (3 Marks)

```

Analysis Box

Compute  $y_i(w^T x_i + b)$ .
Report minimum value.
Verify all values  $\geq 1$ .

def compute_margins(X, y, w, b):
    return y * (np.dot(X, w) + b)

def verify_constraints(margins):
    min_margin = np.min(margins)
    print(f"Minimum functional margin: {min_margin:.4f}")
    print(
        f"All constraints satisfied (margin >= 1): {min_margin >= 0.999}"
    )

...
Minimum functional margin: 0.9996
All constraints satisfied (margin >= 1): True

```

Task B2: Margin Visualization (2 Marks)

Implementation Box

Compute geometric margin width.
Plot margin lines $w^T x + b = \pm 1$.

```
def compute_margin_width(w):
    return 2 / np.linalg.norm(w)
```

```
All constraints satisfied (margin >= 1): True
Geometric margin width: 3.6382159902624247
```

6 Part C — Support Vectors

Task C1: Identify Support Vectors (3 Marks)

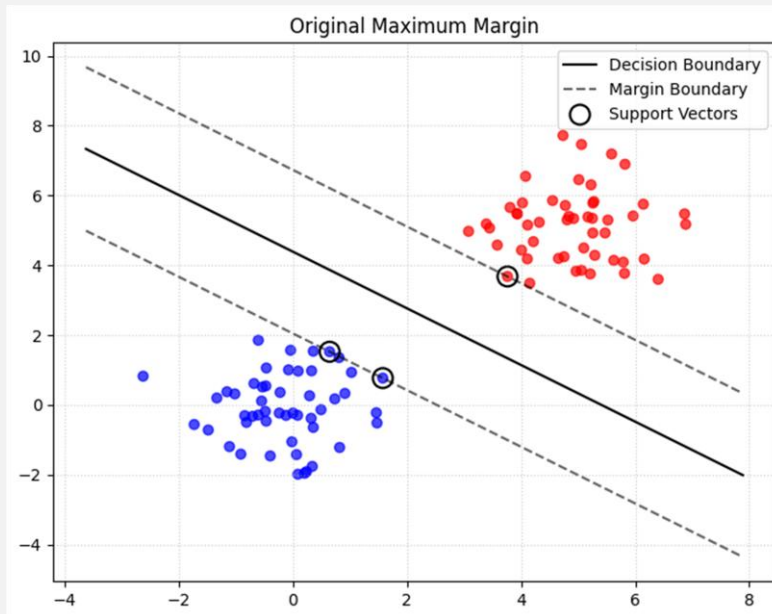
Analysis Box

Print support vector indices and coordinates.
Verify numerically that they satisfy margin equality.

```
def analyze_support_vectors(model, X, y, w, b):
    sv_indices = model.support_
    print(f"Support Vector Indices: {sv_indices}")
    print(f"Number of Support Vectors: {len(sv_indices)}")
    sv_points = X[sv_indices]
    print(f"Geometric margin width: 3.6382159902624247")
    print(f"Support Vector Indices: [ 1  3 73]")
    print(f"Number of Support Vectors: 3")
    print(f"Support Vector functional margins (should be approx 1):")
    print(f"1): [1.00017872 0.99964248 0.99982129]")
```

Task C2: Visualization (2 Marks)**Implementation Box**

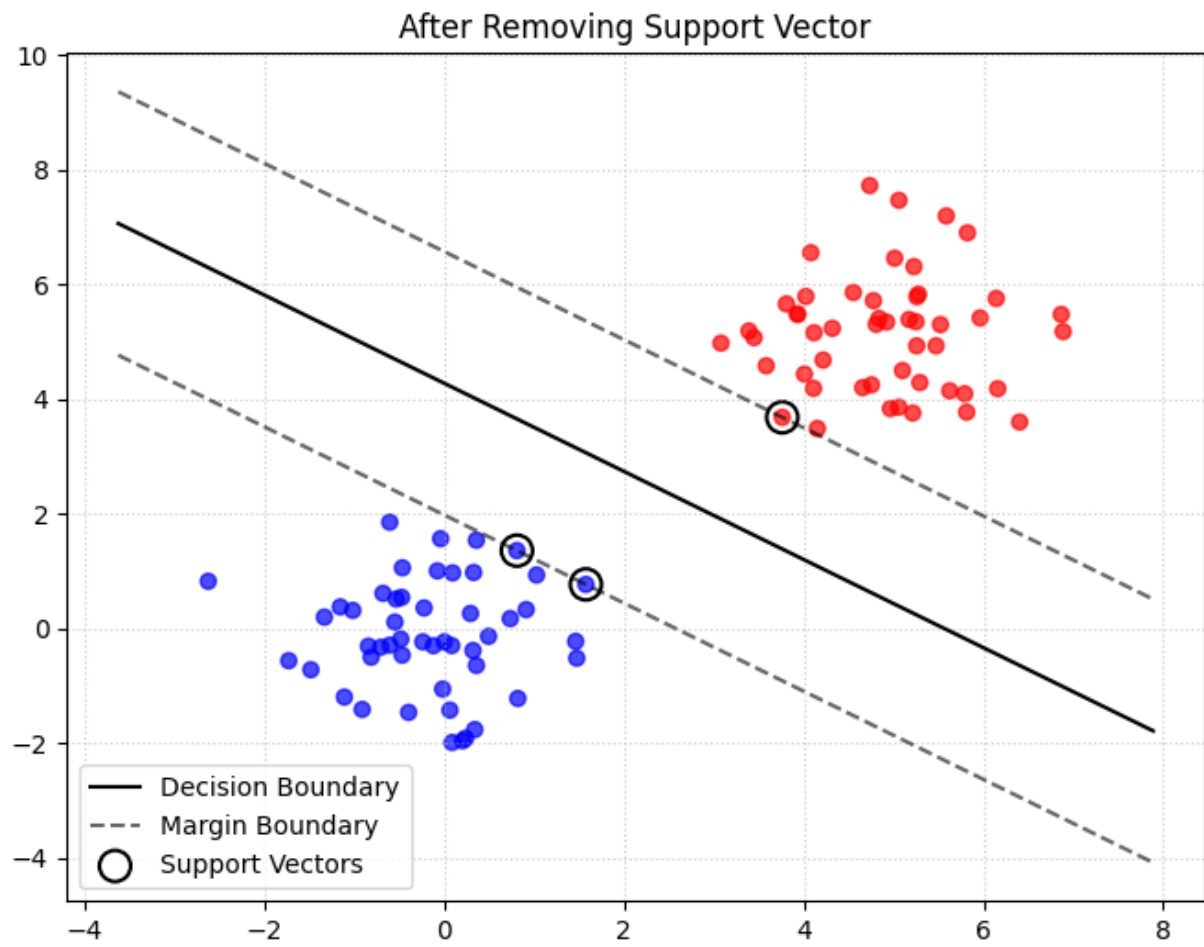
Highlight support vectors on the plot.

**7 Part D — Removing a Support Vector (2 Marks)**

Remove one support vector.

Retrain model.

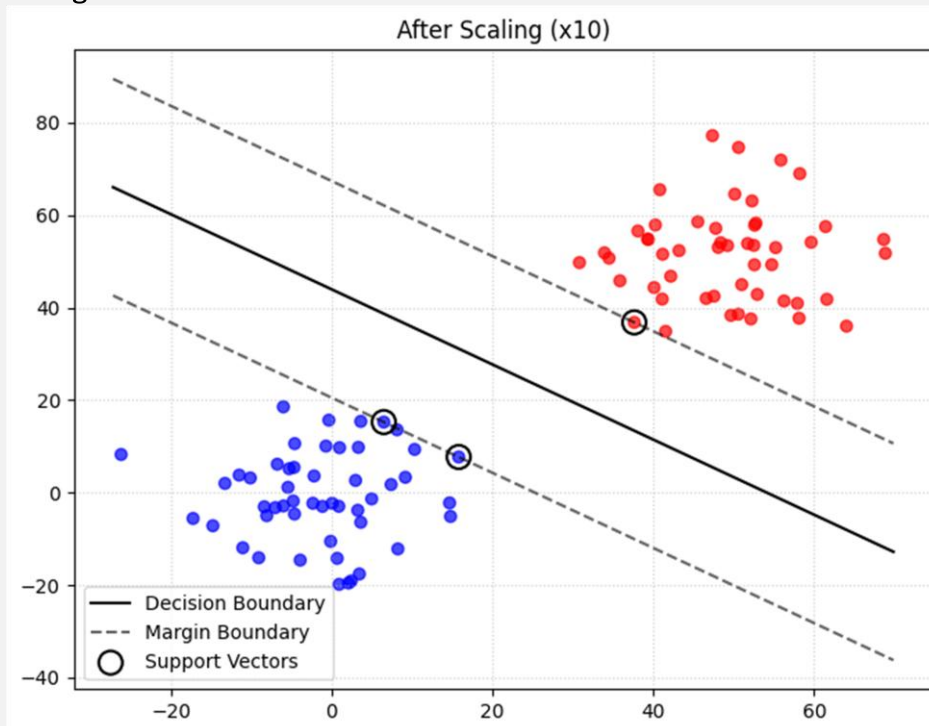
Plot new boundary and compare.



8 Part E — Scaling Experiment (2 Marks)

Experiment Box

Multiply all features by 10. Retrain model.
Compute new margin. Explain
change.



New Margin Width after
scaling: 36.382163412842324

Scaling up your data shrinks the weights,
which stretches the margin width, but
the actual classification stays exactly the
same.

9 Reflection (2 Marks)

Reflection Box**1. Why do only support vectors determine the solution?**

The SVM optimization aims to maximize the empty area (margin) between classes. Because support vectors are the points lying exactly on the boundaries of this margin they act as the constraints that prevent the boundary from moving further. Any point further away already satisfies the safety requirement (>1) and does not push against the boundary.

2. Why does scaling change margin but not classification?

Classification depends on the sign of $w^T x + b$, which remains unchanged if both the data and the weights are scaled proportionally. However, the geometric margin is $2/\|w\|$. When features are multiplied by 10, the weight vector w must decrease by a factor of 10 to keep the functional margin at 1, which in turn increases the geometric margin width.